Informatica

How-To Library

Expose an Application Integration process as an MCP server

© Copyright Informatica LLC 2025. Informatica, Informatica Cloud, and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Expose an Application Integration process as an MCP server

Abstract

To enable AI agents to interact with your Informatica data and assets, you can create an Application Integration process that invokes your Informatica assets and then expose the process as an MCP server.

Supported Versions

- Cloud Application Integration October 2025

Table of Contents

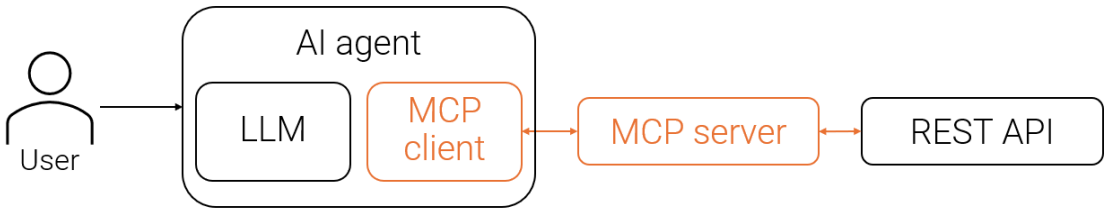
Overview 2
Before you begin 3
Step 1. Create the MCP server configuration 4
Option A: OpenAPI specification 4
Option B: Hand coding 4
Step 2. Run the MCP server 5
Step 3. Test the MCP server 6
Next steps 6

Overview

AI agents that can act on users' behalf are growing in popularity, but to be effective, they need to read data from and perform actions on your live data systems. Model Context Protocol (MCP) is an open protocol that solves this problem by providing a standardized way for large language models (LLMs) to interact with tools, such as data systems and processes. You can leverage MCP to expose your Informatica processes as tools that make your AI agents more accurate and contextual.

Say you want to enable your AI agent to create customer records in Informatica Customer 360 SaaS. If you have a process in Application Integration that creates customer records, you can use MCP to bridge the gap from the process's REST API endpoint to the agent. While MCP can connect to a variety of data systems, this article focuses on REST APIs.

MCP connects AI agents to REST API systems with a client-server architecture. An MCP server defines how agents can interact with a specific REST API, and an MCP client creates a 1:1 connection from an agent to an MCP server. To use MCP to connect a REST API endpoint to an agent, you first create an MCP server and then add a reference to the server in your agent's MCP client. The following image shows the MCP components in a simplified architecture:



When users of the AI agent ask it to perform a task related to the API, the agent uses the client to talk to the server. The server's response comes back to the agent, which then uses its LLM and the new information to write a natural language response to the user.

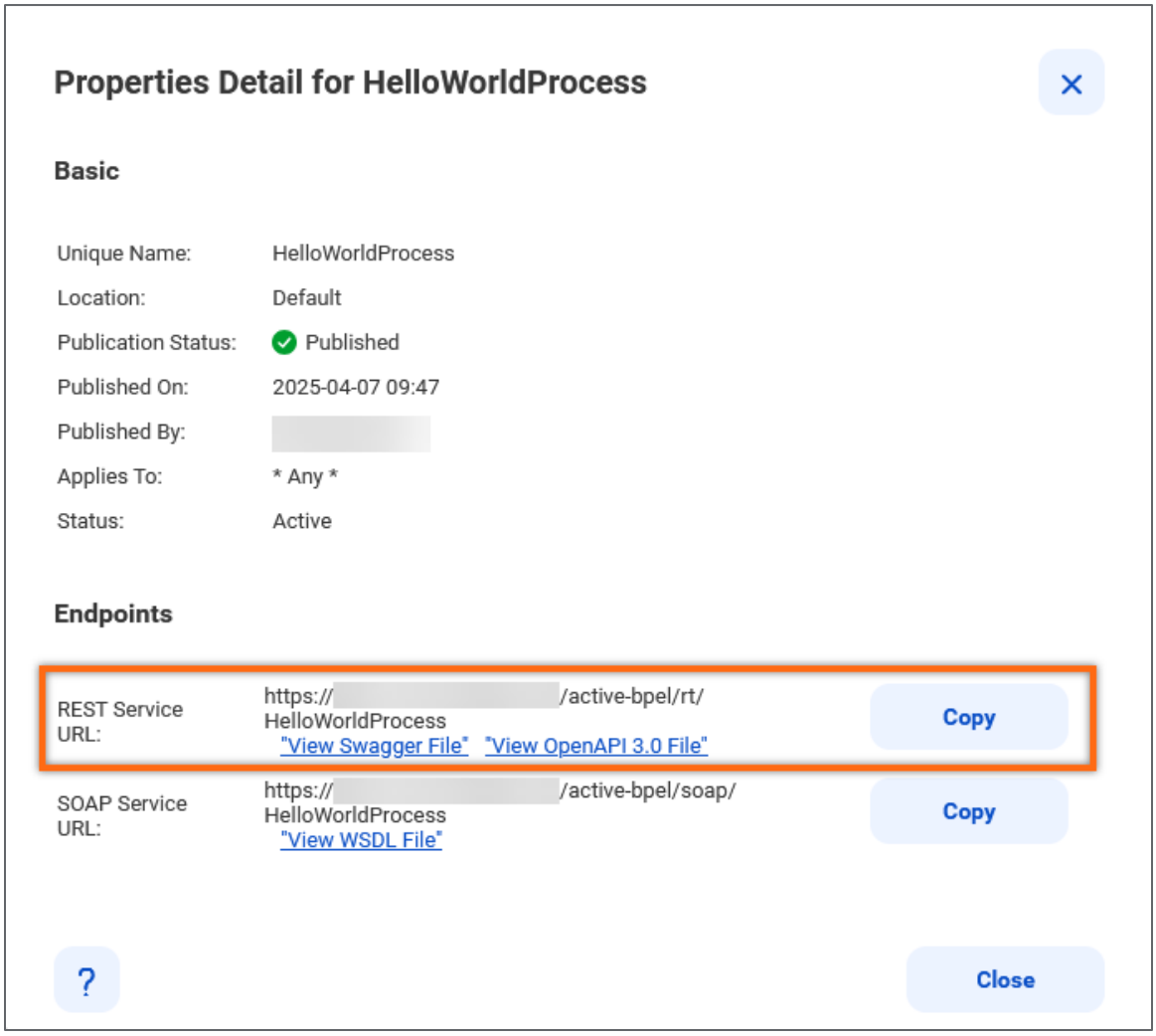
Let’s walk through the process of creating an MCP server for this use case in more detail.

Before you begin

Before you begin, ensure that you have the REST service URL for your Application Integration process.

The process must use the REST/SOAP binding type to generate a REST service URL. You can either configure the process to allow anonymous access or configure authentication within the MCP server.

To get the REST service URL from Application Integration, navigate to the published process and copy the URL from the Properties Detail dialog box. The following image shows the Properties Detail dialog box:



For more information about binding and authentication properties, see “Designing Processes” in the Application Integration help. For more information about published processes, see “Publishing a process” in the Application Integration help.

Step 1. Create the MCP server configuration

You can create an MCP server configuration in a number of ways. In this example, we'll use FastMCP to create a server configuration from an OpenAPI specification and from scratch.

Using an OpenAPI specification offers a quicker conversion from your existing API to an MCP server. Hand coding a server from scratch takes longer but gives you greater control over the server's configuration.

Option A: OpenAPI specification

To create an MCP server using FastMCP and an OpenAPI specification, perform the following tasks:

1. In Application Integration, download the OpenAPI 3.0 file from the Properties Detail dialog box. The OpenAPI specification of a process contains process payload information such as input, output, descriptions, and process objects. For more information, see "OpenAPI JSON specification of a process" in the Application Integration help.

   Make note of where you store the file. We'll use the file path in step 2.

2. Create a *server.py* file that uses the *FastMCP.from_openapi()* class method to convert an OpenAPI specification.

   The following code sample converts an OpenAPI specification to a FastMCP server:

```
import httpx
from fastmcp import FastMCP

# Create an HTTP client for your API
# Set the base URL to the REST service URL from Application Integration
client = httpx.AsyncClient(base_url="<REST service URL>")

# Load your OpenAPI spec
with open("<OpenAPI file path>") as f:
  openapi_spec = json.load(f)

# Create the MCP server
mcp = FastMCP.from_openapi(
   openapi_spec=openapi_spec,
   client=client,
   name="My API Server"
)
```

For more information about FastMCP's integration with OpenAPI, see "OpenAPI" in the FastMCP documentation.

Option B: Hand coding

To create an MCP server using FastMCP and hand coding, perform the following tasks:

1. Create a *server.py* file that instantiates a FastMCP server. For example, add the following code:

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("My API Server")
```

2. Define at least one MCP tool. The following sample tool takes a given name and surname as parameters, posts them through the REST API, and returns the customer ID if successful:

```
@mcp.tool()
def add_customer(given_name: str, surname: str) -> str:

    """Takes the name of the customer to add to Customer 360 SaaS. Returns the ID of the
customer."""

    # Define authentication
    import requests
    headers = {
        "Authorization": "<authorization string>"
    }

    # Define the API request
    url = "<REST service URL from Application Integration>"

    #Create the data object required by the API
    data = {
    "NaturalPerson": {
        "Surname": surname,
        "GivenName": given_name
        }
    }

    # Post the API request
    import sys
    response = requests.post(url, json=data, headers=headers)

    # Handle the response to extract the required payload
    if response.status_code == 200:
        data = response.json()
        if 'RelationId' in data:
                return data['RelationId']

    #Add error handling

    return None
```

For more information about creating tools with FastMCP, see "Tools" in the FastMCP documentation.

Step 2. Run the MCP server

Add code to the MCP server configuration that deploys the server either locally or remotely.

To run the MCP server, perform the following steps:

1. In *server.py*, call the *run()* method on the FastMCP instance. For example, add the following code:

```
if __name__ == "__main__":
    mcp.run()
```

2. If necessary, add arguments to the *run()* call.
    - To deploy the server remotely in a cloud environment, add the argument transport="http" to use Streamable HTTP transport.

      Note: When you deploy the server locally, you don't need to specify a transport type. The default transport type, standard input/output (STDIO), works for local deployment.

However, if you want to make your intent clear, you can add the argument transport="stdio".

- You can set a host, port, and path to bind when using HTTP transport.
- You can set the log level to debug, info, warning, error, or critical.

The following example shows a *run()* call with additional arguments.

```
if __name__ == "__main__":
    mcp.run(
        transport="http",
        host="127.0.0.1",
        port=4200,
        path="/my-custom-path",
        log_level="debug",
    )
```

3. To deploy the server locally, run the server configuration file from the command line. You can run the file using the following Python command:

   python server.py

   Alternatively, if you have FastMCP installed, you can run the file with the following command:

   fastmcp run server.py

4. To deploy the server remotely, refer to the documentation for your cloud provider.

For more information about running a FastMCP server, see "Running Your FastMCP Server" in the FastMCP documentation.

Step 3. Test the MCP server

Use the MCP Inspector to test and debug your MCP server. The Inspector is a visual testing tool for MCP servers.

When you run the Inspector on your local machine, you select a transport type and enter the details to connect to your server. Then use the Resources, Prompts, and Tools tabs to verify that the Inspector correctly discovers the resources, prompts, and tools defined in your MCP server configuration. If there are any issues, fix them in your MCP server configuration, rebuild the server, and reconnect the Inspector to verify the changes.

For more information about using the Inspector, see "Inspector" in the Model Context Protocol documentation.

Next steps

Once your MCP server is up and running, you can add it to the MCP client configuration in your AI agent. When users interact with your agent, they can use natural language requests to get information from or send requests to your REST API. For more information and further instructions, refer to the documentation for your system.